

Make Your Life Easier With Cron



A time-based scheduling utility, Cron is deployed in UNIX-like operating systems. It can be made to run at fixed times, dates or after suitable intervals. It is used to not only update software but also to switch systems on at a particular time and to download mail at fixed intervals.

We often need to schedule tasks at particular intervals of time—for instance, the daily build requirement for software. The developer may write a build script that will check out the source code at midnight and create a build for you. We can also write a Cron job to send out monthly e-newsletters, take backups, upgrade systems and do many other tasks. GNU/Linux provides many utilities to fulfil the above requirements, and Cron is just one of them. So, first, let us understand how Cron works, and then learn more about it by writing some useful Cron jobs.

Cron is a daemon that executes scheduled command(s) at regular intervals. The Cron scheduling utility comes by default with all GNU/Linux distributions. It is started automatically from */etc/init.d* on entering the multi-user run level. The Cron utility uses a Cron table that stores a list of command(s) to be executed after a particular time interval. The Cron table is a plain text file that resides in the */var/spool/cron/crontabs* directory. For each user, there is a separate Cron table (which is named after the accounts in */etc/passwd* file). Command(s) from the Cron table are executed based on the privileges of the user who owns the Cron table. Please note that these Cron tables should not be manipulated directly. GNU/Linux provides the 'crontab' utility that should be used to maintain Cron tables.

Upon startup, the Cron daemon searches its spool area (*/var/spool/cron/crontabs*) and the found Cron tables are loaded into memory. Then the Cron daemon wakes up every minute and examines each command from the stored Cron tables to check which command needs to be executed at the current moment. After successfully executing the command, the output is mailed to the owner of the Cron table. Additionally, each minute, the Cron daemon checks the modification time of its spool area. If it is changed, it identifies the modified Cron tables and reloads them into memory. Thus, there is no need to restart the Cron daemon whenever the Cron table is modified.

The crontab is the other side of the coin. It is a utility that maintains Cron tables for individual users. While manipulating Cron tables, the Cron tab also updates the modification time of the spool area. The two files given below control how the Cron job is executed.

/etc/cron.allow: If this file exists the user must be listed in it, in order to execute the Cron job.

/etc/cron.deny: If the */etc/cron.allow* file does not exist but the */etc/cron.deny* file does, the user must not be listed in the */etc/cron.deny* file in order to execute the Cron job.

If both these files exist, then the */etc/cron.allow* file takes precedence, and if none of these files exist, then only the